

UAS DATA ANALYTICS

Nama : Raymond Widjaja

NIM : 2802474406

Kelompok : 5

1. Review & Evaluation of Data Quality

- a. The main data quality issues that were identified during UTS was the dataset containing many **missing values which consist 19,24%** (107.545) from the total of 558.837 lines across column 'make', 'sellingprice', 'saledate', 'mmr', 'model', 'trim', 'body', 'odometer', 'transmission', 'condition', 'color', 'vin', and 'interior'. Second, **many categorical data had inconsistencies**. Car brand names like "Mercedes-Benz " are written in many ways such as "mercedes," "mercedes-benz," and "mercedes-b" across different rows. Numerical data like 'sellingprice', 'mmr',

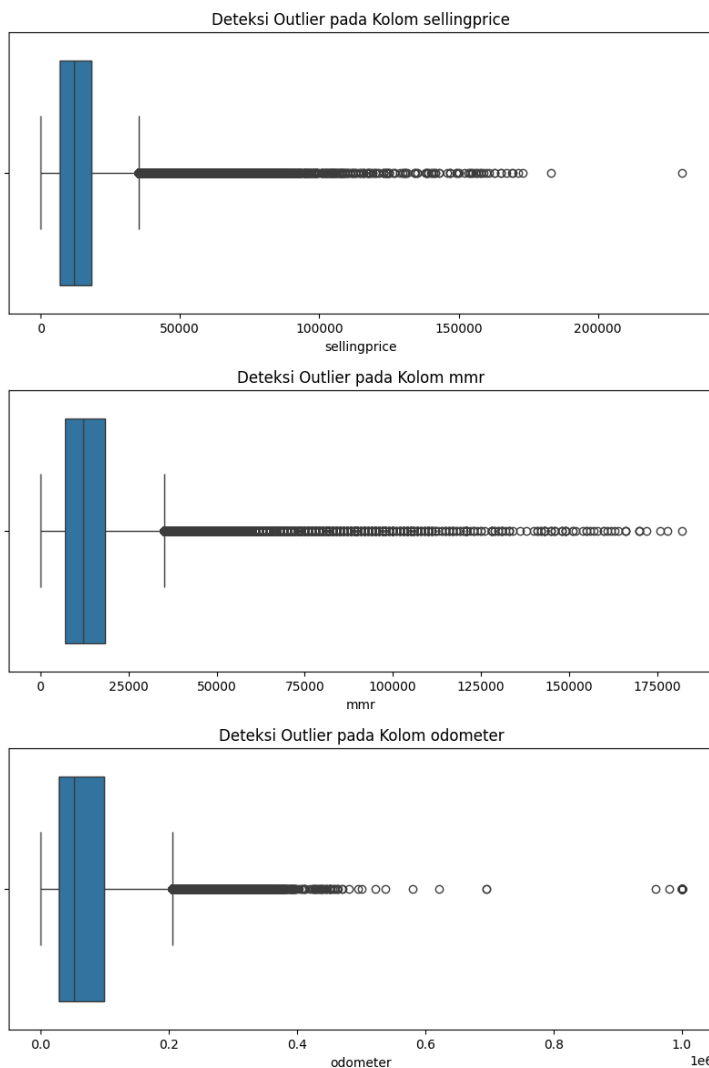
```
Jumlah data asli: 558837
```

```
Jumlah baris duplikat: 0
```

	missing_value	missing_value_percentage
year	0	0.000000
make	10301	1.843292
model	10399	1.860829
trim	10651	1.905922
body	13195	2.361154
transmission	65352	11.694287
vin	4	0.000716
state	0	0.000000
condition	11820	2.115107
odometer	94	0.016821
color	749	0.134028
interior	749	0.134028
seller	0	0.000000
mmr	38	0.006800
sellingprice	12	0.002147
saledate	12	0.002147

and 'odometer' have **extreme outliers** that risk biasing the model.

(Total of data and Total of missing values across column)



(outliers charts of 'sellingprice', 'mmr', and 'odometer')

- b. Through the data wrangling process, we try to improve data quality and prepare it for modeling. **To address the missing value problem**, we can't just drop all the columns, we need to consider our primary column 'mmr', 'models', 'car age', 'trim', 'odometer'. For our numerical primary column we can replace the missing value with the median of the column to minimize data loss. For categorical columns like 'color' and 'interior', we place "N/A" to avoid unnecessary dropping. After that we can drop the rest with `dropna()`.

```
df_clean = df.copy()
#hapus baris dengan kolom kosong
cols1 = ['make', 'model', 'trim', 'body', 'transmission', 'vin', 'condition', 'sellingprice', 'saledate']
df_clean = df_clean.dropna(subset=cols1)

print(f"Jumlah data bersih: {len(df_clean)}")

#ganti data dengan median
df_clean['odometer'] = df_clean['odometer'].fillna(df_clean['odometer'].median())
df_clean['mmr'] = df_clean['mmr'].fillna(df_clean['mmr'].median())
#ganti kolom kosong dengan N/A
df_clean['color'] = df_clean['color'].fillna('N/A')
df_clean['interior'] = df_clean['interior'].fillna('N/A')
```

For standardization of categorical data, we use *str.lower()* and *str.strip()* to help normalize most of it, and for the inconsistencies naming we change it manually.

```
cols = ['make', 'transmission', 'body', 'model']
for col in cols:
    df[col] = df[col].str.lower().str.strip()

koreksi1 = {
    'mercedes': 'mercedes-benz',
    'mercedes-b': 'mercedes-benz',
    'vw': 'volkswagen',
    'land rover': 'landrover',
    'gmc truck': 'gmc',
    'dodge tk': 'dodge',
    'mazda tk': 'mazda',
    'hyundai tk': 'hyundai',
    'ford tk': 'ford',
    'ford truck': 'ford',
    'chevrolet tk': 'chevrolet',
    'chev truck': 'chevrolet',
}
df['make'] = df['make'].replace(koreksi1)

koreksi11 = {
    'f-150': 'f150'
}
df['model'] = df['model'].replace(koreksi11)

koreksi12 = {
    'regular-cab': 'regularcab',
    'land rover': 'landrover',
    'access cab': 'accesscab',
}
df['body'] = df['body'].replace(koreksi12)

koreksi13 = {
    'g coupe': 'coupe', 'elantra coupe': 'coupe', 'genesis coupe': 'coupe',
    'cts coupe': 'coupe', 'cts-v coupe': 'coupe', 'koup': 'coupe',
    'g37 coupe': 'coupe', 'q60 coupe': 'coupe',

    'g sedan': 'sedan',

    'g convertible': 'convertible', 'g37 convertible': 'convertible',
    'q60 convertible': 'convertible', 'beetle convertible': 'convertible',
    'granturismo convertible': 'convertible',

    'cts wagon': 'wagon', 'tsx sport wagon': 'wagon', 'cts-v wagon': 'wagon',

    'minivan': 'van', 'e-series van': 'van', 'promaster cargo van': 'van',
    'transit van': 'van', 'ran van': 'van',

    'crew cab': 'pickup', 'double cab': 'pickup', 'crewmix cab': 'pickup',
    'supercrew': 'pickup', 'quad cab': 'pickup', 'mega cab': 'pickup',
    'extended cab': 'pickup', 'access cab': 'pickup', 'king cab': 'pickup',
    'supercab': 'pickup', 'club cab': 'pickup', 'xtracab': 'pickup',
    'cab plus': 'pickup', 'cab plus 4': 'pickup',
    'regular cab': 'pickup', 'regular-cab': 'pickup', 'regularcab': 'pickup',

    'navigation': np.nan
}
df['body'] = df['body'].replace(koreksi13)

df['transmission'] = df['transmission'].replace('sedan', np.nan)

df['color'] = df['color'].replace('-', np.nan)
df['interior'] = df['interior'].replace('-', np.nan)

df['color'] = df['color'].apply(lambda x: np.nan if str(x).isnumeric() else x)
```

For outliers, we apply the IQR method which effectively identifies and manages the outliers by removing value bigger / lower than the upper bound / lower bound.

```
#outliers
cols = ['sellingprice', 'mmr', 'odometer']
print(f"Jumlah data asli: {len(df_clean)}")

for col in cols:
    plt.figure(figsize=(10, 4))
    sns.boxplot(x=df[col].dropna())
    plt.title(f'Deteksi Outlier pada Kolom {col}')
    plt.show()

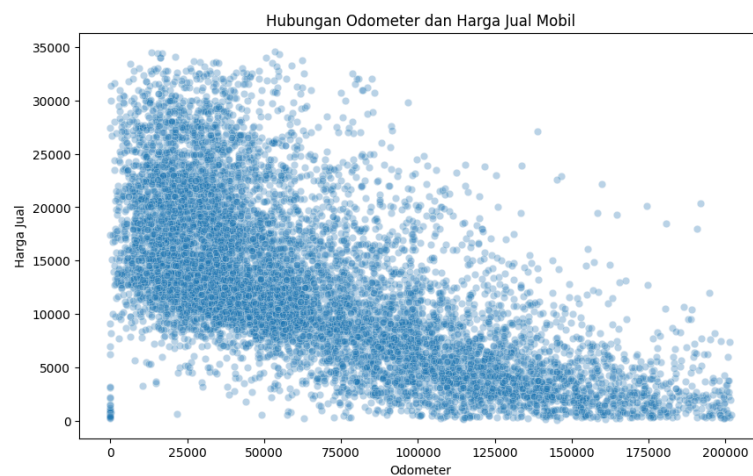
for col in cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    #cari outliers
    outliers = df_clean[(df_clean[col] < lower) | (df_clean[col] > upper)]
    print(f"Jumlah outliers {col}: {len(outliers)}")
    #buang sisa
    df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]

print(f"Jumlah data bersih: {len(df_clean)}")
```

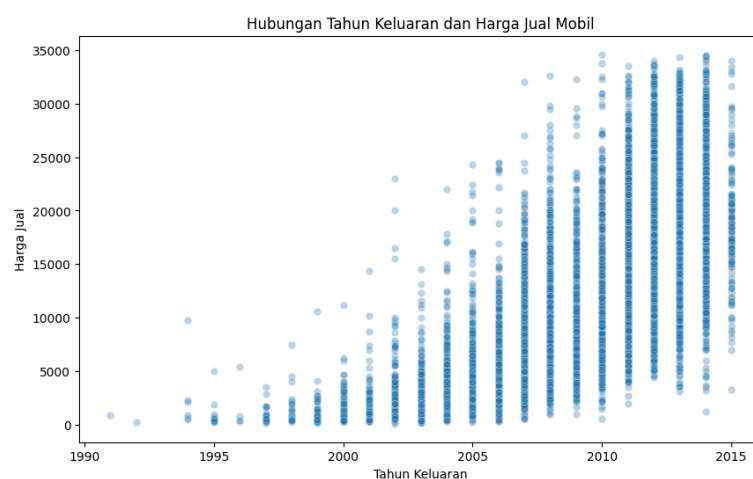
- c. Our data wrangling managed to produce 446.913 lines which is 79,98%, which is good. We avoid dropping important numerical columns by filling it with its median, we place "N/A" on categorical columns on irrelevant columns to avoid dropping too much. We managed to get the missing values and duplicate to 0, and for the outliers it is reduced significantly.

2. Advanced Pattern, Trend, and Relationship Analysis

- a. **Segmented analysis**, this analysis is to see variables relationship deeper between 'odometer' and 'sellingprice', to see deeper we use scatter plot to see if there is a trend on the relationship. Through analysis, there is an inverted curvilinear trend which shows that higher odometer correlated with lower selling prices, even after accounting for mmr (car sales benchmark value). This confirms that: the more a car has been driven, the less it sells for.

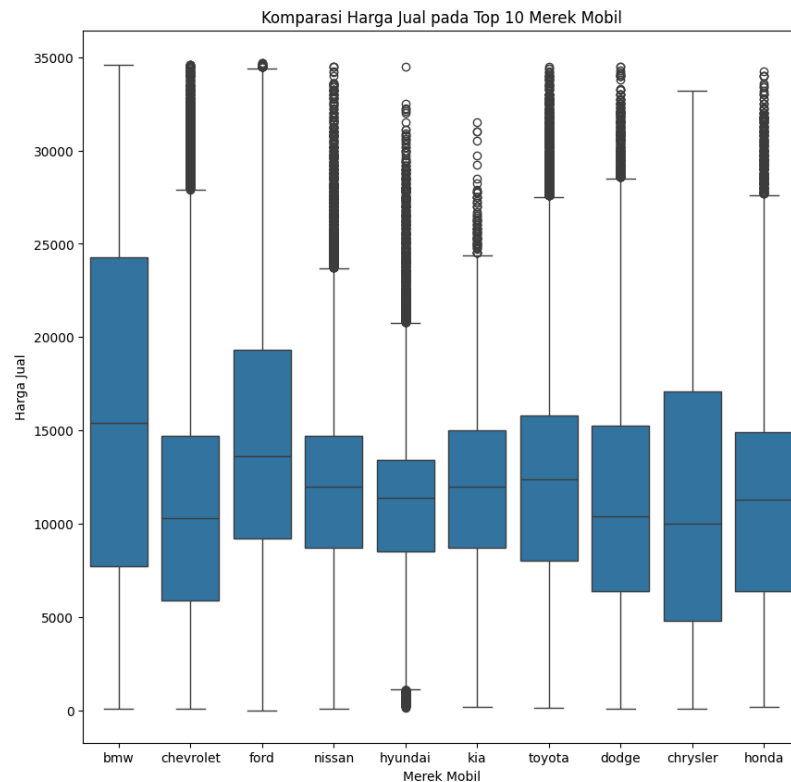


Time-Based Trends, this analysis explored the relationship between 'year' and 'sellingprice' using scatter plot. Using the chart, we can see that there is a positive trend of newer vehicles having higher selling prices which show steady year-over-year depreciation in the used car market.



Subgroup Comparison, the variables to compare in this analysis are 'sellingprice' distribution among 'make' (vehicle brand) using boxplot, we only show the top 10 so we can analyze it much more easily rather than showing everything. There are variations of patterns among subgroups, "bmw" showing a higher and stable

median selling price, not like “chervolet” and “dodge” have a wide range and numerous outliers. This chart tells us that top seller car brands like “bmw”, ”chrysler”, ”ford” have lower whiskers (touching \$0) which mean some had been sold with price lower than MMR, which means the company needs to investigate more on those data.



b. Using *mi_score* (mutual information) we can see which columns have the most impact on the target variable that will be predicted, which is ‘sellingprice’. Here is the list:

- i. mmr (1.660231) The baseline car standard price.
- ii. model (0.515187)
- iii. car_age (0.410801)
- iv. trim (0.402646)
- v. odometer (0.386676)
- vi. condition (0.247342)
- vii. make (0.171087)
- viii. state (0.094388)
- ix. body (0.077163)
- x. interior (0.038538)
- xi. color (0.029011)
- xii. transmission (0.002108)

‘mmr’ is the most impactful since it’s the baseline of car sales price.

c. These relationships match real-world automotive market dynamics which is meaningful, for example : **MMR (Manheim Market Report)**, the baseline market value for a car. It works as a centralized reference point based on actual wholesale transactions, setting the fair market value that dealers and buyers negotiate from.

Car Age & Odometer, show vehicle's economic lifecycle. As a vehicle ages and its mileage increases, it will become obsolete and degraded which makes people less interested to buy causing price loss. **Model & Trim**, they capture consumers' perceived value. Some models have stronger reputation for reliability & status like BMW and Mercedes-Benz models while trim levels separate standard vehicles from premium ones, for example like leather interior, most powerful engine options, advanced safety technology, etc. . People pay more for reliability and “add-on” which the data confirm that these 2 columns are significant. **Condition**, this holds a relatively high *mi_score* but it is subjective and it is already represented by ‘car_age’ and ‘odometer’. **Make**, it is already represented by model and trim which capture the vehicle's worth more precisely, its impact is low. **State**, location does affect demand in specific cases, like 4WD sells better in snowy regions, but across nationwide dataset, it offers no consistent price pattern so it doesn't really impact selling price. **Body**, same as ‘make’ , is already represented by model, and model already implies it, example : A Ford F-150 is inherently a pickup truck, so model captures this information first. **Interior & Color**, Both are subjective. They reflect buyers' personal taste, they don't change the core selling price. **Transmission**, it's already being represented by ‘model’ & ‘trim’, so its impact is low.

3. Evaluation of Analytical Methods and Metrics

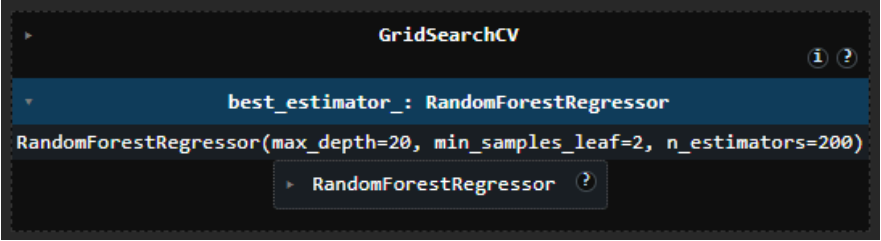
- a. Some analytical techniques or models used during each phased :
 - i. EDA, during it we use **Univariate analysis** and **bivariate analysis** in it. In Univariate analysis we use function *value_counts()* for each categorical column & boxplot for each numerical column, with this we can see the data's distribution and outliers clearly. For bivariate analysis there are 3 relationship:
 1. ‘year’ & ‘sellingprice’ using scatter plot, this is to see the trend from the relationship
 2. ‘odometer’ & ‘sellingprice’ also using scatter plot
 3. ‘make’ & ‘sellingprice’ using boxplot, which is great for numerical & categorical relationship because we can see the median, spread, and outliers of the subgroup
 - ii. **Mutual Information Regression** (*mi_score*), this is to see how much each variable can influence the ‘sellingprice’ value of a car, and it is useful for capturing non-linear relationships rather than using Pearson for linear relationship only. Our *mi_score* only retained the column with the value above 0 so we can focus analyzing informative relationships.
 - iii. **Random Forest Regressor**, the model we used, is a machine learning method of building many decision trees, each trained on a random subset of data, then averages their predictions for final output. We use this because it very well resists overfitting and captures non-linear relationships between features.
 - iv. Hyperparameter tuning is needed to reach maximum output and efficiency to train our model because our data is relatively large and the device specification might bottleneck the process. First, we do **manual tuning** in which we set the parameters manually based on intuition & prior knowledge from the internet. This is the parameter :

```
rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=10,
    max_features=None,
    min_samples_split=3,
    min_samples_leaf=5
)
```

Manual tuning is not efficient and it is really hard to find the best tuning for it so we use **GridSearchCV**. It will try every parameter that is given in the grid, for example like : $max_depth = [20,30]$ & $min_samples_leaf = [1,2]$ they have 4 combinations (2x2), and the training data will be split into the given folds, in this case is 3 ($cv=3$) which means 2 folds for training and 1 for validation, repeating until every fold has been tested, then all the 3 folds combinations are averaged into a single score / combination which it will pick the highest score among them as the best tuning combination. GridSearchCV is good to test every combination within the grid because the best combination is guaranteed to be found, although the downside is the computation load and time depends on the dataset size and feature. Here are the parameters and the best parameters it found :

b. The metrics used in the model are :

```
param_grid = {  
    'max_depth': [20, 30],  
    'min_samples_leaf': [1, 2],  
    'n_estimators': [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]  
}  
  
# hasil sebelum tuning  
# prediksi train  
MAE : 566.35  
RMSE : 836.77  
R2 : 0.9872  
# prediksi test  
MAE : 885.32  
RMSE : 1339.62  
R2 : 0.9672  
)  
  
grid_search.fit(x_train_selected, y_train)
```



The screenshot shows a Jupyter Notebook interface. The top part displays the parameter grid and the results of the fit method. The bottom part shows the best estimator found, which is a RandomForestRegressor with max_depth=20, min_samples_leaf=2, and n_estimators=200.

(evaluation before tuning)

- i. **MAE (Mean Absolute Error)**, for measuring average value difference between predicted and actual selling price in dollars. MAE treats all errors equally regardless of size. From our evaluation before tuning, we can see that MAE of 885.32 which means its predictions are off by \$885 on an average.
- ii. **RMSE (Root Mean Squared Error)**, same as MAE to measure average value difference but it is squared first before averaging then square root. This is for penalizing heavily for large errors making it sensitive. Before tuning RMSE is 1339.62 which means some predictions miss by a wider margin.
- iii. **R² (Coefficient of Determination)**, to measure the target feature ('sellingprice') its prediction accuracy on a scale from 0 - 1. Our R²

score is 0.9672 mean the model is 96.72% accurate in predicting the selling price

Here is the result of all configuration evaluation :

Configuration	MAE	RMSE	R^2
Baseline	885.32	1339.62	0.9672
Manual	894.92	1347.61	0.9668
GridSearchCV	875.99	1327.35	0.9678

The use of these 3 metrics are important to show how well the model fits, R^2 show the model accuracy, MAE & RMSE translate performance into price value. MAE gives the average error, RMSE shows a large error exists. This shows that GridSearchCV makes the best result rather than manual tuning.

- c. 1.) Small margin improvement can be seen in the evaluation metrics, the model accuracy from the baseline and GridSearchCV tuning is not so different (only gain 0.06%) this suggests that the model has already done a good job predicting with default parameters. The GridSearchCV we use has significant limitations, the small grid combination (4 combinations) limited how much improvement was possible but making the grid combination much more varied has its downside like higher computational cost and time. 2.) The parameter for manual tuning has lower accuracy than the baseline, which means intuition or prior knowledge from the internet can make an underperformed model if the constraints are too restrictive for this dataset.

4. Exploratory Data Analysis (EDA)

- a. The insights :
 - i. Use 'mmr', 'model', 'trim', 'odometer', and 'condition' as the selling baseline, since 'mmr' has the highest *mi_score* (1.660), you can standardize pricing around 'mmr' along with other variables to support the price. This is to make consistent baseline prices.
 - ii. Flag vehicles with its selling price that fall anomalously below mmr, using 'price_diff', after seeing the 'make' & 'sellingprice' boxplot, we can see the top brand car like "bmw", "kia", "ford" has lower whisker almost reaching \$0 which means some car sold below mmr but this need to be confirm with "price_diff".
- b. These recommendations will help stakeholders set an objective price baseline. Dealers & auction houses will have an objective price baseline using the model prediction rather relying on intuition or experience alone . Inventory managers can prioritize on flagged vehicles with large 'price_diff' instead of checking manually. This hopes to speed up the negotiation process and reduce the risk of underpricing or overpricing.
- c. potential risk :
 - i. This model is trained on data from a specific period of time, so it may not predict well if there are market shifts (fuel prices, EV adoption, economic downturns)

- ii. The model depends on 'mmr' since mmr has the highest *mi_score* and it is a market driven benchmark, any bias in mmr can make the model learn from the bias. If the mmr somehow "wrong", the model will still learn from it as if it is "ground truth"
- iii. Model can't predict the "very high" mileage & value since the outliers were deleted through IQR.